Alicia McMurchie
3/2/2026
Foundations Of Databases & SQL Programming
Assignment 06

Database Abstraction Layers

Introduction

In a relational database, data engineers and database administrators may use abstraction layers and SQL commands to grant or deny access to tables and views in order to protect production data and improve data usability. There are several different types of abstraction layer that can be used in a relational database, including views, which are saved SELECT statements; user-defined functions, which are saved SQL code that can perform calculations and return results; and stored procedures, which are saved SQL code that can be used to perform a number of tasks including more complex procedures like inserting data into or updating tables. Use of abstraction layers can save time, prevent leaks of sensitive data, and provide a more consistent and reliable user experience when working with data in a relational database.

Use Cases of SQL Views

Views in a relational database are saved SQL SELECT statements that return a specific results set. There are two main types of view: base views and reporting views. Base views show the underlying data in a table without modification, and are created to protect the underlying data by providing an abstraction layer that a data engineer can work with instead of working directly with the data in a table. Reporting views are typically more complex, and are created to present data for reporting purposes. They may include SELECT statements with additional optional clauses such as WHERE, ORDER BY, and various types of join in the FROM clause.

In the example below, base views are created for Categories and Products tables (Figure 1).

```
CREATE VIEW vCategories
    WITH SCHEMABINDING
        AS SELECT CategoryID, CategoryName
            FROM dbo.Categories;
GO

CREATE VIEW vProducts
    WITH SCHEMABINDING
        AS SELECT ProductID, ProductName, CategoryID, UnitPrice
            FROM dbo.Products;
GO
```

Figure 1. Creation of base views for Categories and Products tables, including schema binding.

All of the columns are listed rather than using a SELECT * statement. These base views also include schema binding, which prevents columns in the tables from being modified to the extent that the base views will break.

In order to prevent users in the Public group from attempting to use data in the underlying tables, and instead encourage use of the base tables, the Public group is denied SELECT access on the tables, and granted SELECT access on the base views (Figure 2).

```
DENY SELECT ON Categories TO [public];
DENY SELECT ON Products TO [public];
GO

GRANT SELECT ON vCategories TO [public];
GRANT SELECT ON vProducts TO [public];
GO
```

Figure 2. SQL commands to deny access to Categories and Products tables to the Public group, and grant access to the Categories and Products base views to the Public group.

After base views are created, reporting views can be created that select data from the base views (Figure 3). In this example, a reporting view is created showing the Categories and Products base views joined together and ordered by the Category Name, followed by the Product Name, both in the default ascending order.

```
CREATE VIEW vProductsByCategories
        AS
            SELECT TOP 10000 CategoryName, ProductName, UnitPrice
                FROM vProducts JOIN vCategories
                    ON vProducts.CategoryID = vCategories.CategoryID
                        ORDER BY CategoryName, ProductName;
GO
```

Figure 3. A reporting view created to show data in the Categories base view joined to the data in the Products base view.

The ORDER BY clause is not technically intended for use in views or functions in Microsoft SQL Server, but data engineers can use a workaround by selecting more than the maximum number of rows, allowing the reporting view to display the data as defined by the SELECT statement.

Types of Abstraction Layer

While views are a very commonly used type of extraction layer, data engineers and database administrators may also use functions or stored procedures. Each has specific advantages and

unique behavior that should be considered when deciding when which type of abstraction layer to use.

Views

Views are simply saved SELECT statements. They are useful for creating a layer of protection between data tables and users via creation of base tables, and prevention of failures in applications or reporting systems when the underlying tables experience changes via reporting views. For example, if a table includes street address, city, and state columns that are later combined into a single column, reporting views can continue to break that combined column into individual granular columns as needed. Reporting views are also helpful for saving complicated queries on the data, so that users do not need to write out complex SQL code every time a specific query is needed.

Views can present data from a table or tables with new columns generated that do not exist in the tables from which they select data, and unneeded or sensitive columns hidden. This is useful in that user groups can be granted access only the views showing the data that they need. (Tutorialspoint, https://www.tutorialspoint.com/sql/sql-using-views.htm, 2026)(External Site)

View do not, however, account for parameters, and they do not allow users to execute other actions on data tables, such as adding, modifying, or deleting rows of data.

Functions

There are three main types of function: system functions, table function, and scalar function. System functions are those inherent to SQL server, which cannot be changed by the user, such as GETDATE and @@TOTAL_ERRORS. Table functions return a results table. Scalar functions, as their name suggests, return scalar values, meaning a single resultant value. (Learn Microsoft 365, https://learn.microsoft.com/en-us/sql/relational-databases/user-defined-functions/user-defined-functions?view=sql-server-ver17, 2026)(External Site).

Unlike views, functions can include parameters. In table functions, use of parameters is analogous to WHERE clauses, and in fact, a view with a WHERE clause can often take the place of a table function with a parameter. Parameters are very useful in scalar functions, where they can be used to complete calculation, including when populating new columns. The example on the next page shows a very simple scalar function to multiply two values (Figure 4).

```sql
CREATE FUNCTION dbo.MultiplyValues(@Value1 FLOAT, @Value2 FLOAT)
 RETURNS FLOAT
 AS
  BEGIN
   RETURN(SELECT @Value1 * @Value2);
  END
GO
```

Figure 4. Creation of a simple scalar function to multiple two values.

This function can then be used to populate a new results table, in this case multiplying the unit price in a Products base view by the count in Inventories base view to populate a new column called TotalValue (Figure 5).

```sql
SELECT InventoryID, InventoryDate, vProducts.ProductID, UnitPrice, [Count],
    dbo.MultiplyValues(Count,UnitPrice) AS TotalValue
        FROM vProducts JOIN vInventories
            ON vInventories.ProductID = vProducts.ProductID;
```

Figure 5. A SELECT statement using a user defined scalar function to multiply Count and UnitPrice columns of Inventories and Products base views together to create a new TotalValue column.

The results table shows the new TotalValue column in addition to the columns selected from the two tables (Figure 6).

	InventoryID	InventoryDate	ProductID	UnitPrice	Count	TotalValue
1	1	2017-01-01	1	18.00	39	702
2	2	2017-01-01	2	19.00	17	323
3	3	2017-01-01	3	10.00	13	130
4	4	2017-01-01	4	22.00	53	1166
5	5	2017-01-01	5	21.35	0	0
6	6	2017-01-01	6	25.00	120	3000
7	7	2017-01-01	7	30.00	15	450
8	8	2017-01-01	8	40.00	6	240

Figure 6. The results table of a SELECT statement that includes a user-defined scalar function to populate a new column, TotalValue.

User-defined functions can be used to define constraints when initially creating a table, as well. (Learn Microsoft 365, https://learn.microsoft.com/en-us/sql/relational-databases/user-defined-functions/user-defined-functions?view=sql-server-ver17, 2026)(External Site).

Stored Procedures

Like views and functions, stored procedures are portions of saved SQL code. Like functions, they can also contain parameters. What sets a stored procedure apart from a function or a view is its ability to save other transactions, such as inserts and deletes on a table. This allows data engineers to store and execute complicated SQL code by typing EXECUTE followed by the name of the store procedure as they defined earlier.

The following example shows a stored procedure created to delete rows from an Inventories table if the product in a row has a Category ID of 2 (Figure7).

```
CREATE PROCEDURE pDeleteCondimentsFromInventories2
    AS
        BEGIN TRY
            BEGIN TRAN
                DELETE
                FROM Inventories
                    WHERE ProductID IN (SELECT ProductID FROM Products WHERE CategoryID = 2);
                IF(@@ROWCOUNT > 36) RAISERROR('More than 36 rows are being changed.
                Only a max of 36 rows should be deleted.', 15, 1);
            COMMIT TRAN
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRAN
            PRINT ERROR_MESSAGE()
        END CATCH;
GO
```

Figure 7. The SQL code used to create a stored procedure to delete rows from an Inventories table if the product in that row as a CategoryID of 2.

A user can either use the CREATE PROCEDURE command, or, if they want to use shorthand, they can simply use CREATE PROC. Likewise, they can execute the stored procedure by using EXECUTE followed by the stored procedure name as shown below, or they can just use EXEC and the stored procedure name (Figure 8).

```
EXECUTE pDeleteCondimentsFromInventories2;
```

Figure 8. The SQL code to execute the user-created stored procedure to delete specific rows from the Inventories table.

Microsoft SQL Server also includes system stored procedures, which begin with a prefix “sp_”. Commonly used system stored procedures include sp_help, which returns information about any object in the database, and sp_addmessage to define error messages. (Learn Microsoft 365, https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-

addmessage-transact-sql?view=sql-server-ver17, 2026) (External Site) Because “sp_” precedes system stored procedures, user-created stored procedures should not use this prefix.

Summary

Abstraction layers are important tools for data engineers and database administrators to protect data, streamline workflows, and improve data processing. Views can be used to save SQL SELECT statements, which can help prevent unintended changes to the underlying tables, avoid sharing sensitive or unnecessary data with specific groups, and create detailed reporting views that show specific queries. Functions can be used to return either tables of results or single results, and leverage use of parameters. Stored procedures can be used to define reusable portions of SQL code including more complex transactions. While each method of abstraction has different use cases, in combination, all contribute to a more robust, easy-to-use, and secure database.